Supplementary Material for *Auditing File-Level Static Analysis of Agent Workflows: Extraction Fidelity and Policy Applicability*

Abstract

This supplement discharges the forward references made by the main paper, carries the positioning table displaced from it by the page limit, and adds three reviewer-facing methods documents that the main text has no room for. Every numeric claim below is annotated with the committed artifact path it comes from, or with the script that was executed to produce it. Where a promised item cannot be produced honestly from the artifacts, we say so explicitly and explain why rather than substituting a number of unclear provenance; Sections 1 and 6.2 each contain one such disclosure, and Section 2 states its central result as an explicitly *conditional* theorem whose hypothesis the implementation does not verify.

Contents

Artifact conventions. Paths are relative to the repository root. `corpus/real_world/` holds the mining study's committed JSON; `scripts/` holds the executable analyses. The submitted `reproduce_all.sh` regenerates HGP-1 before every downstream summary, rebuilds matched-set and confidence outputs, and finishes with `validate_submission_claims.py`, which fails if any primary artifact reverts to the withdrawn 3/119 policy union, the pre-exclusion 187-flag set, or the collision-contaminated fidelity set.

1 The Orthogonal Attribute Ontology and its Per-Framework Mapping

Promised at `sections/02_system.tex`, paragraph *An orthogonal attribute ontology*: "The full vocabulary and its per-framework mapping appear in the supplement."

1.1 The vocabulary as implemented

The vocabulary below is transcribed from `src/[pkg]/graph/model.py` (the frozensets `VALID_EFFECTS`, `VALID_CAPABILITIES`, and the `back_edge` field of `GraphEdge`), not from the paper's prose, so the two are guaranteed to agree.

Table 1: The *effect* set: observable side effects a node's execution may have on the world or on shared state. Non-exclusive—a node may carry any subset, including the empty set. Source: `src/[pkg]/graph/model.py`, `VALID_EFFECTS`.

Effect	Definition (verbatim from the implementation)
`read`	reads external data / state (queries, retrieval, fetch)
`write`	writes/persists data (db insert, file write, state store)
`execute`	runs code / commands (code exec, shell, kubectl, restart)
`communicate`	sends messages outward (email, chat, API notify, publish)
`financial`	moves money or places trades (trading/banking/payment)
`delete`	destroys/removes data or resources

Table 2: The *capability* set: structural abilities a node possesses, orthogonal to `NodeKind`. A single node may hold several; the implementation's own worked example is an LLM node that also invokes tools and routes, i.e. `("llm","invokes_tool","routes")`. Source: `src/[pkg]/graph/model.py`, `VALID_CAPABILITIES`.

Capability	Definition (verbatim from the implementation)
`llm`	performs an LLM/model inference call
`invokes_tool`	invokes one or more bound tools
`human_pause`	pauses for human input / approval
`routes`	makes a branching/routing decision over successors
`state_update`	mutates shared graph state
`subgraph`	delegates to / expands a nested subgraph

The edge back-edge flag. `GraphEdge` carries `back_edge: bool = False` alongside `kind`. `EdgeKind.LOOP` is retained for backward compatibility, but the orthogonal flag is the credible encoding: a conditional loop is (`kind=CONDITIONAL`, `back_edge=True`), which the single `EdgeKind` label cannot express. Back edges are computed in `scripts/build_corpus.py` (`_back_edges`) by a classical DFS—an edge whose target is on the recursion stack—so the flag is exactly the set of loop-closing edges, not a name heuristic.

Relationship to the exclusive kinds. The single-label vocabularies remain in the model as the compatibility view and are what every check still keys on: `NodeKind` ∈ {ENTRY, EXIT, TOOL, LLM, ROUTER, HUMAN, SUBGRAPH, PASSTHROUGH} and `EdgeKind` ∈ {DIRECT, CONDITIONAL, PARALLEL,

LOOP}. Note that PASSTHROUGH is a first-class `NodeKind` in the implementation even though earlier drafts of the paper's formal vocabulary omitted it.

1.2 Per-framework mapping

Table 3: Per-framework mapping into the exclusive `NodeKind`/`EdgeKind` view, transcribed from the four runtime extractors under `src/[pkg]/graph/extract/`. "sy" marks synthesized sentinels (elements fabricated by the extractor, provenance `origin=synthesized`).

Framework	Mapping
LangGraph	`START`→ENTRY; `END`→EXIT; `ToolNode` / declared tool binding→TOOL; interrupt/human-named node→HUMAN (name heuristic, so `confidence=may`); conditional-branch source→ROUTER (heuristic); otherwise LLM. Edges from `add_conditional_edges`→CONDITIONAL; from `add_edge`→DIRECT. Nodes and `add_edge` edges are `origin=runtime`, `confidence=exact` unless the kind was guessed.
CrewAI	`Task` with bound tools→TOOL, else LLM; hierarchical-process manager→ROUTER (sy); `__entry__`/`__exit__` sentinels (sy). Task *ordering* is inferred from the crew's task list, so the chaining edges are `origin=runtime`, `confidence=may`; manager fan-out edges are CONDITIONAL and synthesized.
AutoGen	`UserProxyAgent`-like participant→HUMAN; `SelectorGroupChat` selector→ROUTER; other participants →LLM; `__entry__`/`__exit__` (sy). Participants are `confidence=exact`; the transition relation reconstructed from team semantics (round-robin ring, selector fan-out) keeps `origin=runtime` with a weaker confidence, and hook-up edges to the sentinels are `confidence=heuristic`.
Google ADK	`SequentialAgent`/`ParallelAgent`/`LoopAgent` →SUBGRAPH (`confidence=exact`) with children expanded; `LongRunningFunctionTool`/confirmation callback→HUMAN; agent with declared tools→TOOL; else LLM. `SequentialAgent` children are chained by DIRECT edges, `ParallelAgent` children by PARALLEL edges, and `LoopAgent` closes an EDGEKIND.LOOP edge from the last child back to the first (or a self-loop for a single child). All composition edges are `origin=runtime`, `confidence=may`, because the ordering is inferred from the composite's semantics rather than declared edge-by-edge.

1.3 Disclosure: the effect/capability layer is not populated by the extractors

We must be precise about what is implemented, because the main text's phrase "its per-framework mapping" could be read as a claim we cannot support.

A repository-wide search for assignments to the `effects` and `capabilities` fields outside the model definition returns exactly two non-test sites, both in `scripts/build_corpus.py`. **None of the four per-framework extractors under `src/[pkg]/graph/extract/` populates `effects` or `capabilities`, and none sets `back_edge`.** Every graph produced by extraction—runtime or AST, curated or mined—therefore carries the dataclass defaults `effects=()`, `capabilities=()`, `back_edge=False`. The layer is, as the model's own comment puts it, "a purely additive descriptive layer" that "does not disturb `NodeKind`/`EdgeKind` or any check logic".

The only mapping into the ontology that actually runs is the one applied when the *curated*

corpus is authored (`scripts/build_corpus.py`, functions `_tag_node` and `_effects_for`), and it is framework-independent:

- **Capabilities** are a deterministic function of `NodeKind` alone (`_CAPABILITY_BY_KIND`): LLM→`("llm",)`, TOOL→`("invokes_tool",)`, ROUTER→`("routes",)`, HUMAN→`("human_pause",)`, SUBGRAPH→`("subgraph",)`, and ENTRY/EXIT/PASSTHROUGH→`()`. Consequently `state_update` is in the vocabulary but is *never assigned* by any code path in the repository; it is currently vocabulary-only.
- **Effects** are a keyword match over a node's *declared tool names* (`_EFFECT_KEYWORDS`, 30 keyword rules), e.g. `delete/remove/purge/drop`→`delete`; `payment/trading`→`financial` (with `banking`→`financial,write`); `smtp/email/notify/social_media`→`communicate`; `execute/pytest/kubectl/restart/rollback`→`execute`; `insert/dall/canva`→`write` (with `cms`→`write,communicate`); and a read block (`search`, `query`, `fetch`, `retriev`, `scrap`, `parse`, `vision`, `finance`, `bloomberg`, `brokerage`, `market_data`, `bureau`, `verify`, `analytics`, `survey`, `competitor`, `scan`, `validator`)→`read`. Genuinely ambiguous tool names are deliberately left with no effect.

Remark 1 (Consequence for the paper's claims)**.** *Because the ontology is populated only on authored graphs, and only from declared tool names, it inherits exactly the blindness the main text diagnoses: a node that invokes a side-effecting tool inside its body with no declared binding receives the empty effect set. This is the same* `tool→llm` *kind error that dominates LangGraph's residual extraction error, viewed through a different field.* ***A genuine per-framework mapping of framework constructs onto effects and capabilities during extraction is not implemented and is future work.*** *Table 3 documents the per-framework mapping that* is *implemented—the one into the exclusive kinds—and should be read as such.*

2 Event-Trace Soundness: A Conditional Theorem

Promised at `sections/02_system.tex`, paragraph *Soundness requires an event-complete abstraction*: "the full proof is in the supplement."

2.1 Why the existing graph-path lemmas do not suffice

An earlier (arXiv) appendix proves eight lemmas, of which the relevant one (*Static Temporal Soundness*) establishes: if the graph × DFA product reports no violation, then no *path through* G violates the property. The main text now says plainly that this is not the guarantee we need. The gap is not a technicality. A graph path is a sequence of *nodes*; a policy is evaluated over a sequence of *events*. A node whose body issues a tool call that appears nowhere in its declared bindings contributes an event that the graph language does not contain, so a graph that over-approximates every path can still under-approximate the event language. Path containment and event containment are logically independent, and the machinery only ever establishes the former. This section therefore states and proves the event-level result directly, and is explicit that its principal hypothesis is *assumed*, never checked.

2.2 Setting

Definition 1 (Event alphabet and concrete traces)**.** *Fix a finite event alphabet* Σ*. An* event *is a tool invocation with its identity (and, where the policy inspects them, its arguments), a node-emitted*

event, or the distinguished termination marker. For a workflow program P, let $\mathrm{Traces}_{\mathrm{ev}}(P) \subseteq \Sigma^*$ *be the set of finite event traces emitted by* terminating *executions of P, under whatever scheduler, model, and environment P is deployed with. Non-terminating executions contribute no member of* $\mathrm{Traces}_{\mathrm{ev}}(P)$.

Definition 2 (Graph event language)**.** *Let $G = (V, E, v_0, V_f)$ be the extracted graph and let $\lambda : V \to 2^{\Sigma^*}$ assign to each node the set of finite event sequences the abstraction permits that node to emit. For a tool node v with declared binding set $T(v)$, the checker takes $\lambda(v) = T(v)^*$—every finite invocation sequence over the declared tools,* including the empty sequence*—which is the conservative closure described in the main text. For a non-emitting node, $\lambda(v) = \{\varepsilon\}$. Let $\Pi(G)$ be the set of* maximal *finite paths of G from v_0: paths ending at a node in V_f or at a node with no successor. Then*

$$\mathcal{L}(G, \lambda) \;=\; \{\, w_0 w_1 \cdots w_k \;\mid\; v_0 v_1 \cdots v_k \in \Pi(G),\; w_i \in \lambda(v_i) \text{ for } 0 \le i \le k \,\} \;\subseteq\; \Sigma^*.$$

Definition 3 (Policy and its DFA)**.** *A policy φ is an LTL_f formula in the DSL fragment of the main text. Its compiled deterministic automaton is $\mathcal{A}_\varphi = (Q, \Sigma, q_0, \delta, F)$ with accepting set $F \subseteq Q$ and violation set $Q_{\mathrm{viol}} \subseteq Q \setminus F$ defined as the* doomed *states, i.e. those from which no state in F is reachable. A finite trace $\sigma \in \Sigma^*$ is* DFA-accepted *iff $\hat{\delta}(q_0, \sigma) \in F$; by construction of the LTL_f-to-DFA compilation, σ is DFA-accepted iff $\sigma \models \varphi$ in the finite-trace semantics of De Giacomo and Vardi [2].*

Assumption 1 (Event-trace conservatism)**.** $\mathrm{Traces}_{\mathrm{ev}}(P) \subseteq \mathcal{L}(G, \lambda)$.

Assumption 2 (Compiler correctness)**.** *For every $\sigma \in \Sigma^*$: $\hat{\delta}(q_0, \sigma) \in F$ iff $\sigma \models \varphi$.*

2.3 The theorem

Theorem 1 (Conditional event-trace soundness)**.** *Let P be a workflow, G its extracted graph with labelling λ, and φ a policy with DFA $\mathcal{A}_\varphi$. Suppose*

(H1) Assumption 1 holds: $\mathrm{Traces}_{\mathrm{ev}}(P) \subseteq \mathcal{L}(G, \lambda)$*; and*

(H2) every $\sigma \in \mathcal{L}(G, \lambda)$ is DFA-accepted, i.e. $\hat{\delta}(q_0, \sigma) \in F$.

Then every concrete finite terminating execution of P satisfies φ.

Proof. Let e be any concrete finite terminating execution of P and let $\sigma_e \in \Sigma^*$ be its event trace. By definition of $\mathrm{Traces}_{\mathrm{ev}}(P)$ we have $\sigma_e \in \mathrm{Traces}_{\mathrm{ev}}(P)$. By (H1), $\sigma_e \in \mathcal{L}(G, \lambda)$. By (H2), $\hat{\delta}(q_0, \sigma_e) \in F$, so σ_e is DFA-accepted. By Assumption 2, $\sigma_e \models \varphi$. As e was arbitrary, every concrete finite terminating execution of P satisfies φ. □

The proof is deliberately trivial. That is the point: once event-trace conservatism is granted, soundness is immediate, and *all* of the technical content of the guarantee sits in (H1). The remainder of this section is about what the implementation does and does not do towards (H1) and (H2).

2.4 What the implementation discharges: (H2)

(H2) is the hypothesis the product construction actually establishes, and it does so by a standard argument, which we give for completeness.

Lemma 1 (The product decides (H2))**.** *Define the product state space $V \times Q$ with initial state (v_0, q_0) and transition relation*

$$(v, q) \longrightarrow (v', q') \quad \textit{whenever} \quad (v, v') \in E \;\; \textit{and} \;\; q' = \hat{\delta}(q, w) \;\; \textit{for some } w \in \lambda(v).$$

Let R be the set of product states reachable from (v_0, q_0). If (i) no $(v, q) \in R$ has $q \in Q_{\text{viol}}$, and (ii) for every $(v, q) \in R$ with v a termination point (i.e. $v \in V_f$ or v has no successor) and every $w \in \lambda(v)$ we have $\hat{\delta}(q, w) \in F$, then (H2) holds.

Proof. Take $\sigma \in \mathcal{L}(G, \lambda)$. By definition $\sigma = w_0 \cdots w_k$ for some maximal path $v_0 v_1 \cdots v_k \in \Pi(G)$ and choices $w_i \in \lambda(v_i)$. Define $q_i = \hat{\delta}(q_0, w_0 \cdots w_{i-1})$ for $0 \le i \le k$, where the empty prefix gives q_0. An induction on i shows $(v_i, q_i) \in R$: the base case is $(v_0, q_0) \in R$, and the step from i to $i + 1$ follows $(v_i, v_{i+1}) \in E$ while consuming $w_i \in \lambda(v_i)$, hence reaches (v_{i+1}, q_{i+1}). Now v_k is a termination point because the path is maximal, so condition (ii), applied to $w_k \in \lambda(v_k)$, gives $\hat{\delta}(q_k, w_k) = \hat{\delta}(q_0, \sigma) \in F$. Since σ was arbitrary, (H2) holds. Condition (i) is not needed for this implication; it is the earlier-warning mechanism—a reachable doomed state witnesses a bad prefix, and the checker reports `may_violate` at the first event that makes every extension violating rather than waiting for termination. The two mechanisms are disjoint by construction, since $Q_{\text{viol}} \cap F = \emptyset$. □

Remark 2 (Why $\lambda(v) = T(v)^*$ and not $T(v)$)**.** *The closure over* all *finite invocation sequences, including ε, is what makes (H2) robust to the two things static extraction cannot decide about a tool node:* which *of the declared tools fires, and* whether any *fires. Restricting $\lambda(v)$ to single invocations would make $\mathcal{L}(G, \lambda)$ smaller and (H2) easier to satisfy—and unsound, since a real execution invoking a declared tool twice would fall outside it. The closure is the reason a false alarm is possible and a false proof is not,* given *(H1).*

Remark 3 (Divergent cycles)**.** *Lemma 1 quantifies over* maximal finite *paths. If the product has a reachable cycle on which an* **F***-obligation stays pending, the checker returns* `inconclusive` *rather than enumerating the terminating traces that thread the cycle. Universal satisfaction over the finite terminating traces of a finite graph is decidable, so this is an engineering-scope choice and not a semantic limitation; it makes the checker weaker, never unsound.*

2.5 What the implementation does *not* discharge: (H1)

Proposition 1 (Exact provenance does not entail (H1))**.** *Let* `region_is_provenance_exact` *mean that every traversed edge and every visited node carries* `exact` *provenance. Then* `region_is_provenance_exact`(G) *does not imply* $\text{Traces}_{\text{ev}}(P) \subseteq \mathcal{L}(G, \lambda)$.

Justification. Exact provenance is a statement about *how each recorded element was obtained*—this edge was read from a framework declaration rather than guessed—and quantifies only over elements that were recorded. It says nothing about elements that were never recorded. Concretely: let v be a node whose body calls a tool t directly, with no declared binding. Every edge incident to v may be read exactly from `add_edge`, and v may have $T(v) = \emptyset$ recorded exactly, so `region_is_provenance_exact` holds; yet $\lambda(v) = \emptyset^* = \{\varepsilon\}$ while the concrete execution emits t, so $\sigma_e \notin \mathcal{L}(G, \lambda)$ and (H1) fails. Nothing in `region_is_provenance_exact` inspects node bodies for unrecorded events. □

This is not a hypothetical counterexample. It is the `tool`→`llm` kind error that dominates LangGraph's residual extraction error in the main paper's fidelity study (LangGraph node-kind

accuracy 0.670; Table 4), and the same error hides side-effecting tools from the risk-aware gate in 32 of 119 sampled workflows (source: `corpus/real_world/gt_triage_results.json`, via `corpus/real_world/error_decomposition.json`).

How (H1) is actually obtained. In the implementation, (H1) is discharged in exactly one way: **by assertion**. The caller passes `assume_trace_conservative`, asserting that the graph is an authored abstraction whose nodes emit no events outside their declared bindings. Provenance qualifies witnesses but is not a proof of (H1) and never authorizes `safe` by itself. Accordingly:

- On the **curated corpus**, authorship supplies the assertion, and every pruning number in the main text is sound *relative to that assertion*, which the paper labels as such.
- On **mined graphs**, the caller supplies no event-coverage assertion, so the gate withholds `safe` outright. No false proof is emitted, because no proof is emitted.
- In both cases a `may_violate` finding is never suppressed by the gate, so the gate can only lose precision, never soundness.

Remark 4 (The missing piece, stated as such)**.** *Establishing (H1) independently requires a body-level effect analysis certifying that a node emits no events outside its declared bindings—for example, a sound over-approximation of the callees reachable from each node's body, closed under the framework's dispatch mechanisms.* ***This is not implemented.*** *Theorem 1 is therefore a conditional result whose antecedent the system does not verify, and the honest summary of the guarantee is:* the product is sound relative to an authored abstraction whose event coverage is asserted by the caller*. We state it this way rather than in the unconditional form because the unconditional form would be false of the implementation.*

Remark 5 (Scope of Assumption 2)**.** *Assumption 2 is not verified either, but it is* tested*: the compiler's verdicts are differentially checked against an independent, directly recursive reference evaluator—exhaustively over all traces up to length six for two-atom formulas, and property-based beyond. That is evidence, not proof, and we label it accordingly. Unlike (H1), however, it concerns a component we wrote and can in principle verify; (H1) concerns arbitrary third-party Python.*

3 Per-Framework Fidelity: Clustered CIs and the Unmatched Breakdown

Promised at `sections/03_realworld.tex`, caption of Table *tab:realworld_fidelity*: "Repository-clustered bootstrap CIs and the unmatched breakdown are in the supplement."

3.1 Provenance of these numbers

Point estimates come from `corpus/real_world/matched_fidelity.json`, key `provenance_collisions.collision_free_fidelity` ($n = 106$). Intervals were computed by `scripts/supplement_cis.py` (re-run) into `corpus/real_world/supplement_cis.json`: a non-parametric bootstrap with $B = 10{,}000$ resamples, seed 20260712, resampling *repositories* (the `owner__repo` prefix of each slug) with replacement and taking all of a drawn repository's workflows, which is the clustering the main text promises. As a correctness check, the script's point estimates reproduce `collision_free_fidelity` exactly.

Table 4: Corrected instrument (v2) on the matched, collision-free set, with repository-clustered bootstrap 95% CIs. n = workflows, r = distinct repositories. Source: `corpus/real_world/supplement_cis.json` (`v2:*`); point estimates cross-checked against `matched_fidelity.json` → `provenance_collisions.collision_free_fidelity.v2_corrected`.

Framework	n	r	**Edge P**	**Edge R**	**Kind acc.**
LangGraph	38	29	0.954 [0.865, 1.000]	0.836 [0.695, 0.943]	0.670 [0.606, 0.738]
CrewAI	20	17	0.700 [0.440, 0.944]	0.692 [0.435, 0.939]	0.929 [0.850, 1.000]
AutoGen	30	19	0.966 [0.917, 1.000]	0.778 [0.678, 0.872]	0.890 [0.832, 0.945]
ADK	18	13	0.394 [0.108, 0.678]	0.343 [0.067, 0.636]	0.920 [0.854, 0.979]
Overall	**106**	**78**	**0.814** [0.720, 0.897]	**0.709** [0.613, 0.798]	**0.824** [0.778, 0.868]

Table 5: As-mined instrument (v1) on the same matched, collision-free set, with repository-clustered bootstrap 95% CIs. Only LangGraph differs from Table 4; ADK, AutoGen, and CrewAI are bit-identical between instruments. Source: `corpus/real_world/supplement_cis.json` (`v1:*`).

Framework	n	r	**Edge P**	**Edge R**	**Kind acc.**
LangGraph	38	29	0.954 [0.865, 1.000]	0.678 [0.549, 0.791]	0.659 [0.590, 0.730]
CrewAI	20	17	0.700 [0.440, 0.944]	0.692 [0.435, 0.939]	0.929 [0.850, 1.000]
AutoGen	30	19	0.966 [0.917, 1.000]	0.778 [0.678, 0.872]	0.890 [0.832, 0.945]
ADK	18	13	0.394 [0.108, 0.678]	0.343 [0.067, 0.636]	0.920 [0.854, 0.979]
Overall	**106**	**78**	**0.814** [0.720, 0.897]	**0.652** [0.561, 0.739]	**0.820** [0.773, 0.866]

Table 6: Node detection on the matched, collision-free set with repository-clustered bootstrap 95% CIs (v2 corrected instrument). Node detection is the stable dimension; the framework spread lives in edges and kinds. Source: `corpus/real_world/supplement_cis.json`.

Framework	n	**Node P**	**Node R**
LangGraph	38	0.967 [0.903, 1.000]	0.930 [0.841, 0.992]
CrewAI	20	0.867 [0.760, 0.972]	0.867 [0.760, 0.972]
AutoGen	30	1.000 [1.000, 1.000]	0.955 [0.892, 1.000]
ADK	18	0.713 [0.546, 0.863]	0.595 [0.400, 0.782]
Overall	**106**	**0.914** [0.866, 0.956]	**0.868** [0.810, 0.920]

Reading the intervals. Three observations the main text has no room for. First, ADK's interval on edge recall, $[0.067, 0.636]$, is enormous: it rests on 18 workflows in 13 repositories, and the point estimate should be treated as an order-of-magnitude statement only. Second, CrewAI's edge precision and recall coincide exactly (0.700/0.692 with near-identical intervals) because the CrewAI extractor emits a chain whose length is determined by the task list, so a missing task costs symmetric precision and recall. Third, LangGraph's kind accuracy interval $[0.606, 0.738]$ excludes every other framework's point estimate, which is the statistical form of the paper's claim that LangGraph's residual error is a *kind* error rather than an *edge* error.

3.2 The unmatched breakdown

The main table reports the matched, collision-free set ($n = 106$) because it is the only set on which the two instruments are scored over identical inputs. For transparency we give the two larger, less defensible sets alongside it.

Table 7: The three nested fidelity sets, as-mined instrument (v1). $n = 119$ is the original validation sample as reported before the keying defect was found (120 minus the self-repo graph); $n = 115$ is the v1∩v2 intersection; $n = 106$ additionally removes the 9 provenance-colliding slugs. Sources: `matched_fidelity.json` keys `unmatched_fidelity_for_reference.v1_asmined_n119_style`, `matched_fidelity.v1_asmined`, and `provenance_collisions.collision_free_fidelity.v1_asmined`.

Set	Framework	n	Node P	Node R	Edge P	Edge R
Unmatched (119)	LangGraph	40	0.968	0.910	0.956	0.682
	CrewAI	20	0.867	0.867	0.700	0.692
	AutoGen	35	1.000	0.942	0.957	0.770
	ADK	24	0.711	0.622	0.420	0.382
	Overall	*119*	*0.909*	*0.854*	*0.805*	*0.649*
Matched (115)	LangGraph	40	0.968	0.910	0.956	0.682
	CrewAI	20	0.867	0.867	0.700	0.692
	AutoGen	32	1.000	0.958	0.968	0.792
	ADK	23	0.699	0.606	0.395	0.355
	Overall	*115*	*0.906*	*0.855*	*0.803*	*0.649*
Collision-free (106)	LangGraph	38	0.967	0.912	0.954	0.678
	CrewAI	20	0.867	0.867	0.700	0.692
	AutoGen	30	1.000	0.955	0.966	0.778
	ADK	18	0.713	0.595	0.394	0.343
	Overall	*106*	*0.914*	*0.862*	*0.814*	*0.652*

What the exclusions cost. The overall v1 numbers barely move across the three sets (edge recall $0.649 \to 0.649 \to 0.652$; node precision $0.909 \to 0.906 \to 0.914$), which is the reassuring part. The unreassuring part is that ADK's edge recall falls monotonically, $0.382 \to 0.355 \to 0.343$, while its n falls $24 \to 23 \to 18$. The exclusions are not uniform, and Section 9 quantifies the bias.

Current CI authority. `corpus/real_world/confidence_intervals.json` and `supplement_cis.json` now both use the matched, collision-free $n = 106$ slug set and repository-clustered bootstrap. The first additionally records the self-repository-excluded 186-flag triage and HGP-1 inter-

vals. Their point estimates are checked against `matched_fidelity.json`, and `validate_submission_claims.py` fails if the old unmatched $n = 120/187$ inputs reappear.

4 Full Triage Breakdown for the 186 Flags

Promised at `sections/03_realworld.tex`, paragraph *Flag precision on extracted graphs*: "(full triage breakdown in the supplement)".

4.1 Provenance and the 186-vs-187 discrepancy

The triage universe lives at `corpus/real_world/validated_results.json`, key `triage_details`, which holds **187** records. Exactly one of them has a slug naming our own repository (redacted here for anonymity), suffix `__test_extract_adk`—a graph from our own test fixtures, the contamination the main text describes as detected during revision and excluded from every number. Removing it leaves **186**, and the label totals then reproduce the main text exactly. All tables in this section are computed on the 186; the committed `confidence_intervals.json` was computed *before* that exclusion and reports the 187-flag figures (42.8%/49.7% instead of 42.5%/50.0%), which is why the main text does not cite it here and why those numbers must not be lifted from earlier drafts.

Table 8: Per-check × per-label counts for all 186 triaged flags. Rows are the checks as recorded in the artifact (two rows carry the legacy aliases that `error_decomposition.json` maps to `router_shape` and `human_gate_coverage` respectively). Source: `corpus/real_world/validated_results.json` → `triage_details`, self-repo record excluded.

Check	extraction artifact	intentional	arguable	real defect	Total
`human_presence`	8	93	10	2	113
`reverse_reachability`	25	0	0	0	25
`dead_ends`	24	0	1	0	25
`exit_reachability`	14	0	0	0	14
`tool_declarations`	5	0	0	0	5
`router_shape`	2	0	0	0	2
`router-non-conditional-edges`*	1	0	0	0	1
`sensitive-path-bypasses-human-review`†	0	0	1	0	1
Total	**79**	**93**	**12**	**2**	**186**
Share	42.5%	50.0%	6.5%	1.1%	100%

*legacy alias for `router_shape` (combined: 3 flags). †legacy alias for `human_gate_coverage`.

Consistency with the main text. Table 8 reproduces every headline of `sections/03_realworld.tex`: 186 flags; label shares 42.5%/50.0%/6.5%/1.1%; 2 classified as genuine, both under `human_presence` on sensitive workflows; 12 ARGUABLE; and $113/186 = 61\%$ of flags from the blunt `human_presence` check alone. The 72 structural flags are the sum of the six structural rows ($25 + 25 + 14 + 5 + 2 + 1 = 72$), of which 71 are extraction artifacts and 1 is arguable, with *zero* classified as genuine—the main text's "none of the 72 structural flags was classified as genuine".

4.2 The four-way attribution, by check

Attribution maps each triage label to a cause under the rules recorded in `corpus/real_world/error_decomposition.json` → `_meta.attribution_rules`: `extraction_artifact` and `gt_error` → EXTRACTOR; `intentional` → POLICY-SPEC; `arguable` → LABEL; `real_defect` → ACTIONABLE. (No record in the 186 carries the `gt_error` label, so EXTRACTOR coincides exactly with `extraction_artifact` here.) CHECKER is not separable in this *label-derived* attribution and is folded into POLICY-SPEC; both are on the check side, so the extraction-versus-check comparison is unaffected. Section 5 separates the two with a second, label-independent instrument and finds CHECKER = 1.1% [0.3, 3.8] — i.e. the POLICY-SPEC bucket reported here is ≈ 98% specification error, and the two instruments agree on 93.0% of flags.

Table 9: Four-way attribution by check, with Wilson 95% CIs (%). Source: `corpus/real_world/error_decomposition.json` → `decomposition_186.by_check`. Checks with a single flag are reported for completeness; their intervals are uninformative.

Check	n	Extractor	Policy-spec	Label	Actionable
`human_presence`	113	7.1 [3.6, 13.4]	82.3 [74.2, 88.2]	8.8 [4.9, 15.5]	1.8 [0.5, 6.2]
`reverse_reachability`	25	100 [86.7, 100]	—	—	—
`dead_ends`	25	96.0 [80.5, 99.3]	—	4.0 [0.7, 19.5]	—
`exit_reachability`	14	100 [78.5, 100]	—	—	—
`tool_declarations`	5	100 [56.5, 100]	—	—	—
`router_shape`	3	100 [43.9, 100]	—	—	—
`human_gate_coverage`	1	—	—	100 [20.6, 100]	—
All	**186**	**42.5** [35.6, 49.7]	**50.0** [42.9, 57.1]	**6.5** [3.7, 10.9]	**1.1** [0.3, 3.8]

4.3 The four-way attribution, by framework

Table 10: Four-way attribution by framework, with Wilson 95% CIs (%). Source: `corpus/real_world/error_decomposition.json` → `decomposition_186.by_framework`.

Framework	n	Extractor	Policy-spec	Label	Actionable
LangGraph	111	**65.8** [56.5, 73.9]	28.8 [21.2, 37.9]	5.4 [2.5, 11.3]	—
AutoGen	31	16.1 [7.1, 32.6]	**74.2** [56.8, 86.3]	9.7 [3.4, 24.9]	—
ADK	24	4.2 [0.7, 20.2]	**83.3** [64.1, 93.3]	8.3 [2.3, 25.9]	4.2 [0.7, 20.2]
CrewAI	20	—	**90.0** [69.9, 97.2]	5.0 [0.9, 23.6]	5.0 [0.9, 23.6]
All	**186**	42.5 [35.6, 49.7]	50.0 [42.9, 57.1]	6.5 [3.7, 10.9]	1.1 [0.3, 3.8]

The framework split is the aggregate's whole story. LangGraph is the only framework in which extraction is the majority cause (65.8%), and it contributes 111/186 = 60% of all flags. In the other three frameworks policy misspecification dominates by a wide margin (74–90%), and extraction attribution is at or near zero. This is a direct consequence of the structural-check asymmetry the main text notes: the CrewAI, AutoGen, and ADK extractors emit connected topologies by construction and *cannot* produce a dead end or an unreachable exit, so essentially all of their flags come from the blunt human-presence check. The aggregate 42.5% vs. 50.0% is therefore

not a statement about extraction in general; it is a weighted average of one extraction-dominated framework and three policy-dominated ones.

4.4 By check family, and the symmetric ablation

Table 11: Extraction versus check-side attribution among the 184 non-actionable flags, split by check family, with Wilson 95% CIs (%). Source: `error_decomposition.json` → `decomposition_186.by_check_family`.

Family	n (non-act.)	Extraction-attributable	Check-side
Structural (6 checks)	72	**98.6%** [92.5, 99.8]	0.0% [0.0, 5.1]
Human-gate	112	7.1% [3.7, 13.5]	**83.0%** [75.0, 88.9]

Table 12: The symmetric ablation underlying the main text's "what survives a perfect graph?" paragraph. Source: `error_decomposition.json` → `symmetric_ablation` and `headline`.

Quantity	v1 extraction	Reference (oracle) graph
Workflows	119	119
Total flags	184	122
Flags per workflow	1.55	1.03
Structural flags	71	13
Workflows with ≥ 1 structural flag	26	11
`human_presence` firings	113	106
`human_gate_coverage` firings	0	3

Flags removed by a perfect graph: 62 (33.7% of flag volume). Residual that a perfect graph still produces: 122 (66.3%). Genuine defects recovered only on the reference graphs: 2.

Two details worth stating. First, the ablation's baseline is **184** flags, not 186, because it re-runs the check suite on the 119 workflows rather than replaying the triage records; the two universes differ by two flags and the main text is careful never to pool them. Second, `human_gate_coverage` fires *zero* times on extracted graphs and *three* times on reference graphs—the risk-aware check is not insensitive, it is starved of declared bindings, which is the paper's central diagnosis expressed as a single row.

5 The Survival Test: Separating CHECKER from POLICY-SPEC, and Decomposing Misses

The four-way attribution above is derived from triage *labels*, and those labels cannot isolate CHECKER: the triager was asked "is this a real defect?", not "whose fault is this flag?". This section reports a second, *label-independent* instrument that closes that gap and extends the same logic to false negatives. It is implemented in `scripts/fp_fn_decomposition.py` and emits `corpus/real_world/fp_fn_decomposition.json`. It re-reads no source code: every input is a committed artifact, so the decomposition replays deterministically.

5.1 The rule, stated before it was run

For each triaged flag (workflow s, check C) judged non-actionable:

R1 Re-run C on the source-reconstructed reference graph GT(s). If C does *not* fire, the reconstruction removed the flag ⇒ EXTRACTOR.

R2 If C fires and C is *structural* (`dead_ends`, `exit_reachability`, `reverse_reachability`, `router_shape`, `tool_declarations`) ⇒ CHECKER. A structural check is a claim about topology alone; it has no per-workflow applicability parameter to misfire on. If the topology is faithful and the finding is still not a defect, the decision procedure is wrong for that framework's semantics.

R3 If C fires and C is a *policy* check (`human_presence`, `human_gate_coverage`) ⇒ POLICY-SPEC, except when GT(s) already contains a node of kind `human`, where the check contradicts evidence present in the faithful graph ⇒ CHECKER.

R4/R5 `arguable` ⇒ LABEL; `real_defect` ⇒ actionable (outside the denominator). These precede R1–R3: a disputed label is a label error whatever the graph says.

The validity guard. R1–R3 presume the reference graph is correct for the predicate under test. Our own results say it sometimes is not, and two committed sources record exactly where: (a) the GT triage pass labelled the flag `gt_error`, i.e. the reconstruction misrepresents the source; (b) the verdict turns on a `human` node that the HGP-1 placebo audit found non-discharging. When the guard trips, the survival test is not evidence about the check, and we record REFERENCE-ERROR rather than force the flag onto a side. Reporting it as its own cause is the point: it measures how often the source-reconstructed reference is itself wrong, which on this sample is **5 of 186** flags (2.7%). Suppressing that category would have manufactured 3 spurious CHECKER findings and 2 spurious EXTRACTOR findings.

5.2 False positives: the checker is not the problem

Table 13: Label-independent decomposition of the 186 triaged flags by the survival test, with Wilson 95% CIs (%). Source: `fp_fn_decomposition.json` → `false_positives.over_all_flags`.

Cause	n	**%**	**Wilson 95%**
POLICY-SPEC	94	50.5	[43.4, 57.6]
EXTRACTOR	71	38.2	[31.5, 45.3]
LABEL	12	6.5	[3.7, 10.9]
REFERENCE-ERROR	5	2.7	[1.2, 6.1]
CHECKER	2	1.1	[0.3, 3.8]
Actionable	2	1.1	[0.3, 3.8]

Table 14: The same decomposition split by check family, over the 184 non-actionable flags. Source: `false_positives.by_check_family`.

Family	n	**Extr.**	**Checker**	**Policy**	**Ref./Label**
Structural	72	**94.4%**	2.8%	0.0%	2.8%
Human-gate	112	2.7%	0.0%	**83.9%**	13.4%

Three things follow. First, CHECKER is now *quantified* rather than merely bounded below: 2/186 = 1.1% [0.3, 3.8]. The two instances are the LangGraph `interrupt()` multi-turn pattern,

where a genuinely outgoing-edge-free node is a deliberate re-entry point rather than a dead end, and a LangGraph `ToolNode` whose tools are fetched from an MCP server at runtime and so are statically undeclarable. Both are framework-semantics gaps in the check, and neither is a policy question.

Second, the check-side error the paper reports is therefore almost entirely *specification*, not *logic*: 50.5% POLICY-SPEC against 1.1% CHECKER. The checks do what they say; what they say is wrong for these workflows. This matters for the paper's prescription—writing a better checker is not the lever; instantiating the policy per workflow is.

Third, the two instruments agree. Mapping the committed triage labels directly (`extraction_artifact`→EXTRACTOR, `intentional`→POLICY-SPEC) reproduces the survival test's verdict on **173 of 186** flags, 93.0%. The 13 disagreements are listed in the artifact with their triggering rule; 5 are guard trips, and the remainder are cases where a `human` node present in the reconstruction but absent from the extraction (or vice versa) moves the flag between EXTRACTOR and POLICY-SPEC. This is not independent human validation and is not offered as a substitute for it. It is a weaker but real claim: the headline decomposition does not depend on the LLM triage labels, because a mechanical procedure over graphs alone reproduces it.

5.3 False negatives: the opposite budget

Flag triage is blind to misses by construction. The survival test extends to them directly: hold the policy fixed, vary the graph, and ask which of the 12 confirmed HGP-1 violations the pipeline reports.

Table 15: Why the pipeline misses each of the 12 confirmed human-gate violations, under the as-mined extractor and under the reference graph. Source: `false_negatives`.

Cause of the miss	As-mined	Reference graph
Detected (no miss)	0	1
EXTRACTOR	**12**	—
ABSTRACTION	—	**9**
REFERENCE-ERROR	—	1
CHECKER	0	1
Total missed	**12**	**11**

The as-mined column is unmixed: *no* mined graph declares a sensitive tool at all, so the risk-aware check could not have fired on any of the 12, and every miss is extraction. The reference-graph column is the informative one. Swapping in the source-reconstructed reference recovers **1 of 12**. Of the 11 that remain:

- **9 are** ABSTRACTION: the effect is absent from the *reconstruction* too, because it lives inside a node *body*. No extractor could recover it within the `NodeKind`/`EdgeKind`/declared-tool vocabulary these graphs use; the fix is a richer abstraction, not a better reader of the same one.
- **1 is** REFERENCE-ERROR: the reconstruction omitted tools the source declares statically (the `Harsh1210__llm-opsbot__main gt_error`), so this one *is* recoverable by a better model.
- **1 is** CHECKER: the workflow was cleared on a gate that cannot block—the placebo failure mode, appearing here as a false negative rather than a false positive.

Why this is the paper's sharpest result. The two directions have *opposite* error budgets. Non-actionable findings are check-side (50.5% policy specification versus 38.2% extraction). Misses are model-side (100% extraction as mined). But the reconstruction repairs only 1/12 of the misses, because the missing information sits *below* the abstraction rather than above it. A reader who concluded from the false-positive decomposition that "extraction fidelity is a secondary concern" would draw exactly the wrong lesson: extraction fidelity bounds what the analysis can *see*, and improving the extractor within a body-blind abstraction moves that bound by one workflow in twelve.

This is the same boundary the certification gate meets (Section 2.5). Exact provenance certifies that an *edge* was read correctly; it says nothing about the events inside the node that edge connects. Table 15 is the empirical measurement of that gap: on 9 of 12 source-audited violations, the abstraction is silent about the very effect the policy regulates.

6 Scalability, Extractor Accuracy, and a Modeling-Effort Comparison

Promised at `sections/04_results.tex`: "details, per-framework extractor accuracy on stubs, and a Promela/SMV modeling-effort comparison are in the supplementary material."

6.1 Scalability (re-run)

Table 16: Structural check time versus graph size, **freshly re-run** for this supplement via `scripts/benchmark_scale.py` (median of 10 trials, synthetic graphs at edge density 2.0). Output artifact: `scripts/scaling_results.json`. Monitor compilation and evaluation are size-independent and are reported once below the table.

Requested nodes	Actual nodes	Actual edges	Structural checks (ms)
50	52	104	0.060
100	102	214	0.111
200	202	430	0.221
500	502	1,074	0.592
1,000	1,002	2,128	1.267
2,000	2,002	4,252	2.977
5,000	5,002	10,713	**7.191**

Monitor compilation (5 rules): 0.259 ms. Monitor evaluation (1,000 events × 5 rules): 9.264 ms, i.e. ≈108,000 events/s. Both are independent of graph size.

These numbers are hardware-dependent, and we say so loudly. The same script, same code, and same machine produced 7.611 ms at 5,000 nodes on one invocation and 7.191 ms on the next—a 6% run-to-run spread from nothing but scheduling noise. More importantly, the previously committed artifact recorded 239.838 ms and an earlier arXiv draft reported 104.7 ms at the same size. The three figures span a factor of ≈33, and none of them is wrong: they are three different machines. **Only the asymptotic shape should be read as a property of the system**; the absolute milliseconds are a property of whatever hardware ran the benchmark, and we therefore make no cross-draft comparison of them.

Shape. Over 50 → 5,000 nodes the graph grows 96× in $|V| + |E|$ while check time grows 120×, i.e. very slightly super-linear. The excess comes from quadratic edge scanning in the router-shape and tool-declaration checks, which is invisible at the sizes that actually occur: the mined corpus has a *median of 2 non-sentinel nodes* (`sections/03_realworld.tex`; `corpus/real_world/metadata_v2.json`), so the largest row of Table 16 is roughly three orders of magnitude beyond anything we observed in the wild.

6.2 Per-framework extractor accuracy

Disclosure: the promised "on stubs" measurement is not available. The main text promises "per-framework extractor accuracy on stubs". The harness for it is `scripts/extractor_accuracy.py`, whose defaults point at `corpus/graphs/` (extracted) and `corpus/ground_truth/` (reference). `corpus/ground_truth/` **does not exist in the repository**, and the script exits with "No ground truth files found". The nearest substitute, running `corpus/graphs/` against `corpus/curated/`, returns 1.000 on every metric for all four frameworks—but that is an artifact, not a result: `corpus/curated/` is *authored* by `scripts/build_corpus.py` as literal `GraphNode`/`GraphEdge` constructions, and `corpus/graphs/` is a copy of it differing only in the provenance fields. Comparing them measures a file copy. **We therefore report no stub-accuracy table and record the promise as unmet rather than filling it with a circular** 1.000**.** Producing it honestly requires stub *source files* plus independent reference annotations; neither is committed.

What we can report: accuracy on real mined code. The same script, run against the real-world corpus (re-run: `-corpus corpus/real_world/graphs -truth corpus/real_world/ground_truth`, output `scripts/accuracy_results.json`), gives a genuine per-framework measurement. Note this is the *unmatched* $n = 120$ set including the self-repo graph, so it is a per-instrument diagnostic and not the paper's headline; Section 3 carries the defensible numbers.

Table 17: Per-framework extractor accuracy against the LLM-reconstructed reference graphs (re-run; `scripts/accuracy_results.json`). Unmatched $n = 120$ set. "P"/"R" are macro-averages over workflows.

Framework	n	Node P	Node R	Kind acc.	Edge P	Edge R
LangGraph	40	0.968	0.910	0.647	0.956	0.682
CrewAI	20	0.867	0.867	0.929	0.700	0.692
AutoGen	35	1.000	0.942	0.900	0.957	0.770
ADK	25	0.709	0.602	0.943	0.403	0.367
Overall	**120**	**0.907**	**0.848**	**0.829**	**0.798**	**0.644**

Failure modes. The same run categorises every node-kind mismatch. Over all 120 workflows there are **104** categorised mismatches, distributed as `custom_node_type` **98** (94.2%) — "unrecognized node kinds or custom agent types", which in practice is overwhelmingly the `tool→llm` error — and `naming_heuristic_failure` **6** (5.8%) — "human/router detection by name convention failed". Two further categories exist in the taxonomy (`dynamic_construction`, `version_incompatibility`) and score **0**. This single distribution is the quantitative core of the paper's diagnosis: kind error is not a long tail of framework quirks, it is one failure mode with a 94% share, and it is the one that hides side-effecting tools from the risk-aware gate (Section 2.5).

6.3 Modeling-effort comparison against general-purpose model checkers

The table below is reproduced verbatim from the committed artifact `papers/paper1/generated/comparison_table.tex` (generated by `scripts/generate_comparison_table.py`). It is qualitative by construction and we do not dress it up as a measurement.

Table 18: Comparison with general-purpose model checkers. Source: `papers/paper1/generated/comparison_table.tex`.

Tool	Input	Properties	Modeling	Time	Domain
SPIN [3]	Promela (manual)	Full LTL	High	Fast (small)	None
NuSMV [1]	SMV (manual)	CTL + LTL	High	Fast (small)	None
CBMC [5]	C source	Assertions	Medium	Bounded	None
This work	Auto-extracted	Safety LTL fragment	**None**	$O(\|V\|+\|E\|)$	**Agent workflows**

SPIN, NuSMV, and CBMC require manual translation of agent workflows into their input languages. Our analyzer extracts models automatically from framework APIs, eliminating the modeling step entirely.

What "Modeling: None" does and does not mean, and what is not measured. Two honest qualifications. First, the modeling effort is not eliminated, it is *amortised*: the four per-framework extractors encode framework semantics and were hand-written once, and Section 3 is the measurement of how lossily they do it. What is eliminated is the *per-workflow* manual translation SPIN and NuSMV require. Second, the "Time" column is not a head-to-head benchmark. The repository contains exactly one hand-written Promela model, `corpus/comparisons/email_triage.pml`, and *no committed SPIN or NuSMV run, no timing, and no lines-of-model count for any other workflow.* **A quantitative modeling-effort comparison—person-minutes or lines of Promela per workflow, across a sample—was not performed, and no number for it appears in this supplement.** The comparison above is a capability-and-workflow comparison only.

7 Sampling Design and the Limits of the Corpus

Not promised by the main text, but included because every prevalence number in Section 3 of the paper is bounded by it. The following inlines `papers/paper1/aaai/SAMPLING_DESIGN.md`, which was reconstructed entirely from repository artifacts.

Bottom line. The 922-graph corpus is a **search-ranked convenience corpus**, not a probability sample of GitHub agent workflows. The 119-graph validation set is a **hand-fixed, unseeded, quota-shaped subset** of it, hard-coded into a driver script; no sampler, no seed, and no reproducible selection rule exists in the repository. Inclusion probabilities are therefore not merely unestimated, they are **undefined**. Every framework-share reweighting in the paper is a **weighted descriptive statistic over a convenience corpus**, not a design-based prevalence estimate for GitHub.

7.1 The frame, verbatim

The frame is the set of files returned by 11 GitHub code-search queries, truncated by a per-query result cap and a global repository-list cap, in GitHub's opaque relevance ("best match") order, then

all `*.py` files inside the repositories that survived that truncation. Queries are from `scripts/mine_github_gh.py`:37–49, executed via `gh search code`.

Table 19: The 11 verbatim mining queries. Source: `scripts/mine_github_gh.py`:37–49.

Framework	Query string (verbatim)
LangGraph	`StateGraph add_node language:python`
LangGraph	`StateGraph add_conditional_edges language:python`
LangGraph	`from langgraph.graph add_edge language:python`
CrewAI	`from crewai Crew Task language:python`
CrewAI	`crewai "process=Process" language:python`
AutoGen	`GroupChat autogen language:python`
AutoGen	`RoundRobinGroupChat language:python`
AutoGen	`SelectorGroupChat language:python`
ADK	`SequentialAgent google.adk language:python`
ADK	`from google.adk Agent language:python`
ADK	`ParallelAgent google.adk language:python`

Caps and truncation order.

- **Per-query result cap: 40.** `gh search code ... -limit 40` (`mine_github_gh.py`:52–58; default `-per-query 40` at line 295). Confirmed by artifact: `candidates.json` holds **341** items over 11 queries, and AutoGen alone contributes exactly $120 = 3 \times 40$.
- **No sort key is ever passed**, so `gh search code` returns GitHub's default "best match" relevance ranking—a proprietary, undocumented, non-stationary function. Every subsequent truncation inherits this ordering.
- **Repository-list truncation:** `unique[:max_repos]`, default 120, run at 180, at `mine_github_gh.py`:125 and :231. The unique-repo list is built by first appearance in `candidates.json` order, i.e. in relevance order, then cut. Verified: `repos.lg_crew.json` (180 repos) is set-identical to the first 180 unique repositories of `candidates.json` in candidate order.
- **No per-repository file cap.** `find_workflow_files` (`scripts/scrape_workflows.py`:139–151) walks every `*.py` file under the clone, skipping only dot-dirs, `venv`/`.venv`, `node_modules`, `__pycache__`. The paper's earlier phrase "per-repository caps" was inaccurate: the cap is on the repository *list*, not on files per repository.
- **Framework assignment is a text heuristic**, not a dependency check: a file is labeled with framework f if ≥ 2 of f's marker tokens appear anywhere in it (`scrape_workflows.py`:126–136).

Two mining runs. Run 1 (LangGraph/CrewAI) used the full 341-candidate list with `-max-repos 180`; AutoGen and ADK extraction did not yet exist, so the 180-repo prefix yielded 282 records over 149 repos. Run 2 applied the `-frameworks autogen,adk` filter (`mine_github_gh.py`:326–330) over 139 unique repos—the 180 cap was not binding—for 798 records. Union: 318 candidate repositories, of which 253 yielded ≥ 1 extracted graph. Mining date: **11 July 2026**, established independently in `corpus/real_world/MINING_DATE_FORENSICS.md`.

An undocumented silent de-duplication. Graph filenames are `slug` = ⟨owner⟩`___`⟨repo⟩`___`⟨file stem⟩ (`mine_github_gh.py`:170, :256) written with a plain overwrite. Two files with the same basename in one repository collapse to one graph, last-writer-wins. Extracted records across both metadata files: **1080**; distinct slugs on disk: **930**; hence **150 extracted graphs (13.9%) were silently overwritten**. Worst case: `ed-donner/agents` produced **55** files that all collapsed into the single slug `ed-donner__agents__sidekick`. Section 9 treats the downstream consequence for the fidelity comparison.

7.2 The 119-graph validation sample: no seed, no sampler

The validation sample is not computed anywhere. It exists only as a hard-coded JavaScript array literal: `scripts/wf_run.js`, `const GT` (60 items: LangGraph 40, CrewAI 20) and `scripts/wf_run_v2.js` (60 items: AutoGen 35, ADK 25). Total 120; the ADK self-repo item is dropped downstream, giving 119 (LangGraph 40, CrewAI 20, AutoGen 35, ADK 24) from 87 repositories. **There is no sampler script, no manifest, and no record of how the 120 were chosen.** The absolute paths embedded in those arrays point to a different machine and a differently named repository, so the list was produced outside this repository's history.

Refuted selection hypotheses. Tested per framework against the corpus in mining order, all *negative*: first-k prefix in mining order; first-k after dropping < 3-node graphs; first-k with one graph per repository; top-k by graph size; round-robin over repositories in mining order (best overlap 15/25, ADK). Selected positions are scattered across each framework's whole pool (LangGraph ranks 2–391 of 402; ADK 0–52 of 53), and the sample contains up to 4 graphs from one repository, which rules out one-per-repo designs.

No seed exists. The only seeded sampler in the repository is `scripts/make_annotation_samples.py` (`-seed`, default `20260713`, line 462; `rng = random.Random(args.seed)` at line 485). It draws the *human-annotation* sub-samples *from* the 119, inheriting them as a fixed given, and cannot document how the 119 arose.

Table 20: Achieved sampling fractions (descriptive, not design). Source: `corpus/real_world/metadata_v2.json` and `revision_analyses.json`.

Framework	**Corpus** N	**Sample** n	n/N
LangGraph	400	40	10.0%
AutoGen	359	35	9.7%
CrewAI	113	20	17.7%
ADK	50	24	**48.0%**
Total	**922**	**119**	12.9%

7.3 Why inclusion probabilities are undefined

Four independent reasons, each sufficient alone.

1. **The first-stage selection is a proprietary ranking, not a randomization.** `gh search code` with no `-sort` returns GitHub's "best match" order: undisclosed, non-stationary, and deterministic given the index. Truncating it at 40 gives every candidate an inclusion *indicator* of 0 or 1, not a probability. There is no π_i to compute and no way to bound it.

2. **The frame's denominator is unknown.** GitHub does not publish the size or membership of the code-search index, and no total-hit count was recorded at mining time—`gh_search` (`mine_github_gh.py`:52–70) discards everything except the capped item list. Even a Horvitz–Thompson estimator with guessed weights has no N to expand to.

3. **The second stage compounds the first deterministically.** `unique[:180]` is a hard cut on the same rank order—an entire framework (AutoGen, ADK) fell below it in run 1. That is a selection mechanism correlated with the ranking signal, not with anything exchangeable.

4. **The third stage is a census with a non-random thinning.** Within a surviving repo all `*.py` files are taken (cluster sizes 1 to 55), and the slug-collision overwrite then deletes 150 of them by filename coincidence.

Add the unreconstructible hand-selection of the 119 and there is no level of the design at which a probability model can be attached.

Consequence. No design-based variance estimator is licensed. The Wilson intervals and the repository-clustered bootstrap are legitimate as *model-based uncertainty for the 119 observations under an i.i.d.-within-stratum assumption*, and are the right thing to report, but they quantify **only** sampling noise **within the corpus** and carry **no** coverage guarantee for GitHub. The dominant error term—selection bias of the corpus itself—is unquantified and unquantifiable from these artifacts. For the same reason the finite-population correction an earlier draft applied (`scripts/reviewer_analyses.py`:148, `fpc = 1 - n/N_f`) is *not licensed*, since an FPC presupposes simple random sampling without replacement from N_f; the clustered-bootstrap interval is the defensible one and is the one quoted.

7.4 The reweighting: what it is and is not

`scripts/reviewer_analyses.py`:326–372 post-stratifies each per-framework rate by corpus framework shares, $\hat{p} = \sum_f (N_f/N)\,(d_f/n_f)$, with CIs from a repository-clustered bootstrap within strata at fixed weights. The correct name for this quantity is a **corpus-share-weighted descriptive statistic**: the rate the 922-graph corpus would show *if* each framework's 119-sample rate held across that framework's corpus members. It is a within-corpus standardization that removes the ADK over-representation of Table 20. It is **not** a prevalence estimate for GitHub, and it is **not** post-stratification in the survey sense, which requires known *population* control totals; here the control totals are the corpus's own counts, so the procedure cannot correct any bias introduced before the corpus existed—which is all of the bias.

7.5 The corpus framework mix, and a retracted "correction"

The authoritative framework mix is obtained by counting the `framework` field of each of the 922 non-self-repo graph JSONs under `corpus/real_world/graphs/` directly—the field the extractor writes from the file it actually parsed. That count is:

A retraction. An earlier revision of our own methods audit (`papers/paper1/aaai/SAMPLING_DESIGN.md`, §6.2) asserted that this mix was wrong and that the artifacts implied AutoGen 357 / CrewAI 115. **That assertion was itself the error, and we retract it here.** It was produced by reading framework labels for the *v1* corpus out of the *v2* re-mine records—the defect at `scripts/matched_fidelity.py`:251 documented in Section 9.5—which mislabels exactly the seven

Table 21: Corpus framework mix. Source: direct count of the `framework` field over all 922 non-self-repo files in `corpus/real_world/graphs/`; independently confirmed by `matched_fidelity.json` → `drops.corpus_framework_distribution` and by the hard-coded weights at `scripts/reviewer_analyses.py`:57.

Framework	N	**Share**
LangGraph	400	43.4%
AutoGen	359	38.9%
CrewAI	113	12.3%
ADK	50	5.4%
Total	**922**	100%

slug-collision cases in which the two miners resolved the same slug to different source files. The net effect of those seven, AutoGen −2 and CrewAI +2, is precisely the discrepancy the audit reported and misattributed to a self-repo bookkeeping slip.

No published number changes. We checked the downstream consequence rather than assuming it. The post-stratification weights hard-coded at `scripts/reviewer_analyses.py`:57 are {langgraph 400, crewai 113, autogen 359, adk 50}—the *correct* mix—and `corpus/real_world/reviewer_analyses.json` → `2_post_stratification.corpus_composition` records the same values as actually used. Every post-stratified figure was therefore computed with the right weights throughout, and none of them moves—neither the superseded two-check row (structural 0.23%, policy 1.92%, composite 2.15%) nor the HGP-1 figures the main text's Table 2 now reports (structural 0.23%, policy 12.30%, composite 12.52%), which use the identical weights. The erroneous 357/115 pair never propagated beyond the prose of the audit document and the corresponding row of an earlier draft of this supplement, both now corrected. The sampling fractions in Table 20 are the only numbers in this supplement that changed, and they change in the fourth significant figure (AutoGen 9.8% → 9.7%, CrewAI 17.4% → 17.7%).

7.6 What the design does support

Descriptive claims about this corpus, with full provenance (every graph is pinned to `(repo, SHA, path)` and re-extractable via `scripts/remine_pinned.py`); instrument-fidelity claims, which are properties of the instrument on these inputs and do not require a probability sample; the existence and direction findings (a biased frame makes a *low* observed defect rate more notable, not less, provided no population number is attached); and relative comparisons within the corpus. What it does *not* support is any sentence of the form "X% of agent workflows on GitHub". Finally, the frame itself is **not reproducible**—re-running the 11 queries today returns a different top-40 each. What *is* reproducible is the corpus *given* `candidates.json`/`repos*.json`. Reproducibility claims should be scoped to "the corpus is reproducible from the pinned manifest", never to "the mining is reproducible".

8 The Prospectively Specified Human-Gate Policy (HGP-1)

Included because it is the instrument behind the human-gate audit. The following inlines `papers/paper1/aaai/HUMAN_GATE_POLICY.md`. Machine-readable record: `corpus/real_world/human_gate_audit.json`; script: `scripts/human_gate_audit.py`.

Status and purpose. Prospectively specified: written before any of the 32 candidate workflows were assessed for gate presence, and before any comparison against `human_presence` or `human_gate_coverage` output. Its purpose is to replace a conflated estimand—three cases produced by two different checks over two different artifact populations—with one operational procedure applied uniformly to every workflow in the sample that contains a side-effecting tool at *source* level, so that false negatives outside both flag universes become detectable.

Unit and evidence base. One workflow = one mined source file (a slug in `corpus/real_world/gt_triage_results.json`); denominator $n = 119$. Evidence is the source text under `corpus/real_world/sources/`, or the recorded `repo/sha/path`. Extracted graphs may be consulted as a *navigation aid only*: a verdict must be justified by a source line, never by a graph node kind, because the `tool→llm` error is precisely the failure under audit.

Table 22: (a) What counts as a side-effecting / irreversible tool. Classification is by what the callee actually does in source, not by its name.

Category	Definition and positive examples
`destructive`	Deletes, drops, truncates, overwrites, or irreversibly removes existing persistent state: `DROP TABLE`, `DELETE FROM`, `os.remove`, `shutil.rmtree`, `git reset -hard`, cache purge, resource teardown.
`external_write`	Creates or mutates persistent state outside the process—databases, cloud storage, repositories, ticketing/CRM—or a non-idempotent HTTP `POST/PUT/PATCH/DELETE` to a third-party API: `INSERT/UPDATE/MERGE`, `open(path,'w')` outside a temp dir, `bigquery.insert_rows`, `firestore.set`, `git commit/push`, issue/PR creation.
`exec`	Executes attacker- or model-influenced code or commands: `subprocess.run`, `os.system`, `exec/eval`, `PythonAstREPLTool`, `PythonCodeExecutionTool`, `kubectl apply`, container exec, `docker run` on a shared daemon.
`financial`	Moves money or creates a financial obligation: payment capture, transfer, trade order, invoice issue, refund, charge.
`comms_third_party`	Delivers a message outside the operator's trust boundary: SMTP send, SendGrid/Gmail send, SMS, Slack/Discord channel post, social-media publish, outbound customer webhook.

(b) What counts as a human gate. A control point at which execution *blocks pending a human decision that can prevent the side effect.* Exactly one of: (1) an explicit human checkpoint node—graph node of kind HUMAN, LangGraph `interrupt()`, `interrupt_before=[n]` or `NodeInterrupt`, ADK `LongRunningFunctionTool` or tool-confirmation callback, CrewAI task with `human_input=True`; (2) an equivalent confirmation step in the node body—a blocking read of human input (`input()`, `click.confirm()`, a Streamlit/Gradio button handler, an awaited approval message) *whose negative outcome causes the side-effecting call to be skipped*, with the negative branch present in source; or (3) AutoGen `UserProxyAgent` with `human_input_mode="ALWAYS"` positioned so the human speaks between the proposing agent and the executing agent.

Explicitly not a gate (fixed before case labeling): a generic conversational REPL in which the human supplies the *task* and the agent then autonomously chooses and executes tools (the human authorises a goal, not the specific irreversible action); `human_input_mode="NEVER"` or `"TERMINATE"`; logging, printing, or streaming with no blocking decision; post-hoc review occurring *after* the side

effect has landed; and a gate on some other branch that does not lie on the path to the side-effecting call.

(c) Violation condition. A workflow is a violation iff there exists at least one path from the workflow entry point to an invocation of a side-effecting tool that contains no human gate. Equivalently: the set of human gates does not *dominate* the side-effecting call site. A gate on one branch does not discharge an ungated sibling branch, and one ungated reachable side-effecting tool suffices regardless of how many others are correctly gated.

(d) Exclusions. E1 read-only tools (retrieval, search, `SELECT`, HTTP `GET`, vector query, file read, web scrape); E2 pure computation; E3 in-process state only (Python dict, agent/session state, LangGraph channel state, conversation history); E4 LLM text generation, unless the text is then persisted or transmitted by a separate call; E5 genuine mocks and stubs, the exclusion attaching to the *implementation* not the file's name; E6 constructively sandboxed side effects (writes confined to `tempfile`/`tmp_path`, or execution inside a container created and torn down for that call), with the confinement visible in source.

Explicitly not excluded (fixed before case labeling): N1 demo/toy/tutorial/example status—"it's only a demo" is a statement about intent, not behaviour, and intent is recorded separately without changing the verdict; N2 test files, which require E5 or E6 to be exempt; N3 unconfigured credentials—a send that would fail for want of an API key still counts, since the missing gate is the defect; N4 local filesystem writes outside a temp directory.

(e) Undecidable cases. The verdict domain is exactly {`violation`, `compliant`, `arguable`, `source_unavailable`}. `arguable` is mandatory when (i) the side-effecting callee is imported from a module absent from the corpus and its behaviour cannot be established, (ii) tool dispatch is fully dynamic, or (iii) a candidate gate exists but whether it dominates the call site depends on runtime configuration not fixed in source; an `arguable` verdict must state which applies. **An undecidable case is never recorded as `compliant`**—silent downgrading of uncertainty to a negative is the specific bias this policy exists to prevent. We report violation/119 with a descriptive Wilson interval, plus a sensitivity band whose lower arm counts only `violation` and whose upper arm counts `violation` + `arguable`; both are reported regardless of direction.

8.1 Applied results

All 32 sources were read and none dropped; `Sidreyas__The_Grand_AI_Repo__builder` was missing from `corpus/real_world/sources/` and was recovered from GitHub at the recorded sha `61bb48c6`, so there are **zero** `source_unavailable` cases. One threshold rule was clarified at application time and applied uniformly in both directions: a side-effecting callable counts only if it is agent/LLM-invocable or is called inside a graph node body that executes during a workflow run; script-level setup, teardown, and reporting code is outside the threshold.

- Prevalence over the sample: $\mathbf{12/119 = 10.08\%}$, Wilson 95% $[5.86\%, 16.80\%]$.
- Sensitivity upper arm (`violation` + `arguable` $= 23/119$): $[13.24\%, 27.34\%]$.
- Prevalence among the audited side-effecting workflows: $12/32 = 37.50\%$ $[22.93\%, 54.75\%]$.

Table 23: HGP-1 outcome over the 32 source-level side-effecting workflows. Source: `corpus/real_world/human_gate_audit.json`.

Verdict	Count
`violation`	**12**
`compliant`	9
`arguable`	11
`source_unavailable`	0

HGP-1 supersedes the earlier two-check procedure. HGP-1 is the paper's **primary** human-gate instrument, and the main text and its Table 2 report the 12/119 (10.1%) [5.86, 16.80] figure derived here. It *supersedes* the 3/119 (2.52%) figure an earlier draft reported: of those 3 original cases, HGP-1 retains 1 and finds 11 additional violations, a fourfold upward revision that places the superseded figure *outside* the new interval.

The supersession is a matter of instrument quality, not of preferring the larger number. The earlier figure was produced by two *different* checks (`human_presence` and `human_gate_coverage`) applied to two *different* artifact populations (mined graphs and reconstructed graphs), so it was never a single estimand; and both checks read gate presence off a graph node's *kind*, which is exactly the field the `tool→llm` error corrupts. HGP-1 replaces this with one prospectively specified procedure applied uniformly to source text, with a verdict domain that forbids recording an undecidable case as compliant. The two subsections below—the three violations invisible to *both* earlier checks, and the eight non-discharging "gates" that both checks accepted—are the direct evidence that the superseded procedure was measuring gate *labels* rather than gate *behaviour*.

False negatives. Three genuine violations lie outside *both* flag universes—neither `human_presence` nor `human_gate_coverage` fires on them: `Arhamirfan__Snake-Agents__snake_autogen`, `Itzkuldeep__AgentE-com__main`, and `SAMAR-CODE404__backend__Main`. A further 9 violations are missed by the risk-aware check specifically (11 of the 12 have `human_gate_coverage_failed = false`); those 9 fall inside the `human_presence` universe, but that universe contains 106 of 119 workflows and so carries almost no information.

8.2 Placebo gates: a distinct failure mode

Of the 32, **8** carry a human gate that cannot discharge the obligation, and 5 are outright placebos—a node typed `human` in the *reference* graph whose body can never block execution.

All 7 side-effecting workflows that both existing checks cleared were cleared because the reference graph contained a node typed `human`; on source inspection **none of the 7 is a genuine dominating gate**. The bias is therefore *not* confined to the extractor: the source-reconstructed reference graphs themselves overstate human oversight, because `human` is assigned from node name or framework type rather than from whether the body can block. Any estimand computed over those graphs—including the corrected, risk-aware one—is biased **downward**, systematically rather than randomly. The `human_detection` fidelity metric compounds this: for `Arhamirfan__Snake-Agents__snake_autogen` and `Itzkuldeep__AgentE-com__main` it records tp = 1, recall = 1.0, scoring the extractor as *perfectly* recovering human oversight that does not exist; for `SAMAR-CODE404__backend__Main` it records fn = 6, penalising the extractor for failing to reproduce six placebo gates.

Table 24: The 8 non-discharging gates. Source: `corpus/real_world/human_gate_audit.json`.

Slug	Nature of the gate
`SAMAR-CODE404__backend__Main`	placebo: 6 `*_human_approval` nodes whose body is `proceed = self.approval` against a constructor flag hard-set `True`
`Arhamirfan__Snake-Agents__snake_autogen`	placebo: `UserProxyAgent` with `human_input_mode="TERMINATE"`
`timleow__gym-kfc-daddies__cli_ui`	placebo: `get_user_prompt` is a task REPL, not an approval
`Sidreyas__The_Grand_AI_Repo__builder`	inert: `web_user_proxy` serialized into a config, never executed
`waqasniazi9__voice-agent__builder`	inert: `web_user_proxy` serialized into a config, never executed
`Itzkuldeep__AgentE-com__main`	non-dominating: real `ALWAYS` proxy, but ordered after the executing agent
`Sujas-Aggarwal__langraph-chatbot__v2.6.0`	partial: `confirmation_node` auto-confirms above a similarity threshold
`merdandt__SalesShortcut__websiter_creator_agent`	unverifiable: `request_URL` typed `human`, body absent

Honest limitations of HGP-1. The 11 `arguable` cases are dominated by reason (i): the side-effecting callee lives in a module the mining pipeline never captured (single-file snapshots of multi-file packages). A repository-level corpus would resolve most of them, and several look likely to resolve toward `violation`—most notably `Zen7-Labs__Zen7-Payment-Agent__agent`, a crypto payment/settlement pipeline whose prompts forbid confirmation four separate times and where only the callee body, not the missing gate, is unverifiable. They are held at `arguable` because the policy requires it, not because the evidence is balanced. The 87 workflows screened as having no source-level side effect were not re-read under HGP-1, so upstream screening error can change the 12/119 count in either direction. Finally the audit is **single-rater**; `ed-donner__agents__sequential_agents` (prompt-level confirmation only) and `microsoft__ACV__test_cache_agent` (compliant) are the two calls most likely to move under a second rater.

9 The Slug-Collision Keying Defect and the Matched-Set Methodology

Elaborates `sections/03_realworld.tex`, paragraph *A keying defect, and how we corrected for it.* All numbers here come from `corpus/real_world/matched_fidelity.json` → `provenance_collisions`.

9.1 The defect

Graphs are keyed by `slug` = ⟨repo⟩___⟨file basename⟩, which is not unique within a repository. The artifact's own note states the consequence precisely: `remine_pinned.load_records()` resolves collisions *first-wins*, so the v2 instrument may re-mine a *different file* than v1 did for the same slug. Such a graph pair compares two different programs, not two instrument versions. **76 slugs collide** in total; 51 of them are in the matched corpus and 9 are in the matched fidelity set.

Table 25 lists them. The pattern is visible in the names: repositories that keep one `agent.py` per framework subdirectory.

Table 25: The nine colliding slugs inside the matched fidelity set, with the number of distinct source files that mapped to each. Source: `matched_fidelity.json` → `provenance_collisions.fidelity_collision_slugs`.

Slug	Files collapsed
svpino__ml.school__agent	5
Saimoguloju__AI-Agents__sample_agent	4
lordpython__multi-agent-video-system__agent	4
Astrodevil__ADK-Agent-Examples__agent	3
MikhailMostWanted__Astra__app_team	3
joansantoso__iox-adk__agent	2
shazy89__ai-learning__agent	2
volcengine__veadk-python__main	2
winstonbartlegod__enhanced-multimodal-rag__v1	2

Table 26: Re-keying recovery analysis. Source: `matched_fidelity.json` → `provenance_collisions.rekey_recovery` (the artifact also enumerates all 58 unrecoverable records individually under `unrecoverable_detail`).

Quantity	Count
Colliding slugs	76
Present on disk, v1	58
Present on disk, v2	51
v2 records re-keyable to an exact file path	51
v1 records re-keyable, even only by framework	5
Usable v1–v2 pairs recovered	**0**
Must drop	**58**

9.2 Why zero of the 58 pairs are recoverable

The asymmetry in Table 26 is the whole explanation. The v2 miner records enough provenance that all 51 of its colliding records can be re-keyed to an exact file path. The v1 miner resolved collisions *last-wins without recording which file survived*, so v1's file identity is *structurally* unrecoverable—at best 5 of 58 can be pinned even to a framework. A pair requires both halves; since the v1 half is missing for at least 53 of 58 and under-determined for the rest, **zero usable pairs** can be reconstructed. This is not a matter of effort: the information was never written down. All 58 are dropped, and the released instrument fixes the keying.

9.3 The drop is not uniform: ADK-concentrated bias

Table 27: Framework composition of the fidelity sets before and after the collision exclusion. Source: `matched_fidelity.json` → `provenance_collisions.fidelity_drop_bias`.

Framework	Matched (115)	Dropped (9)	Remaining (106)	Drop rate
ADK	23	5	18	**21.7%**
AutoGen	32	2	30	6.2%
LangGraph	40	2	38	5.0%
CrewAI	20	0	20	0.0%
Total	**115**	**9**	**106**	7.8%

ADK loses more than a fifth of its stratum while CrewAI loses none. The cause is mechanical rather than statistical: ADK repositories disproportionately use per-framework directories containing identically named `agent.py` files, which is exactly the collision pattern. **ADK's row in the main paper's fidelity table is therefore the least stable and should be read as such**—a judgment that Table 4's interval, $[0.067, 0.636]$ on edge recall, independently confirms.

Table 28: Reference-graph confidence of the dropped versus retained graphs. Source: `matched_fidelity.json` → `provenance_collisions.fidelity_drop_bias`.

	high	medium	low
Dropped (9)	8 (88.9%)	1 (11.1%)	0
Matched (115)	75 (65.2%)	32 (27.8%)	8 (7.0%)

The exclusion also removes proportionally *more* high-confidence reference graphs than low (88.9% of the dropped set versus 65.2% of the matched set), so the retained set is slightly lower-quality on average than the set it came from. We report the direction because it is adverse to us; we do not attempt to correct for it, since with $n = 9$ any correction would be noise.

9.4 Effect on the structural-flag comparison

The main text quotes the collision-free row (299 → 229), which is the defensible one. For completeness: 75 workflows are flagged *only* in v1 and 1 *only* in v2, and of the 150 graphs whose structure changed between instruments, 40 changed because of a collision (different file) and 110 because of a genuine extractor change—the artifact separates these under `corpus_structurally_changed`, which is what licenses the main text's causal reading of the corrected-instrument ablation.

Table 29: Structural flags on the matched corpus, with and without the colliding slugs. Source: `matched_fidelity.json` → `matched_structural_flags` and its `collision_free` sub-key.

Set	v1 flagged	v2 flagged
Matched corpus ($n = 904$)	313	239
Collision-free ($n = 853$, 51 removed)	299	229

9.5 A second defect with the same root cause: framework mislabelling

The collision defect has a downstream sibling that we found while preparing this supplement, and it is worth stating separately because it bit our own audit rather than the paper.

`scripts/matched_fidelity.py` built the corpus framework census with

```
corpus_fw = Counter(status.get(s, {}).get("framework") for s in corpus_
v1)
```

at line 251, where `status` is the map of *v2* re-mine records (`load_v2_status`) but `corpus_v1` is the set of *v1* slugs. It therefore labels every v1 graph with the framework of whatever file *v2* resolved that slug to. For a non-colliding slug the two agree and the bug is invisible; for a colliding slug they need not, because v1 resolved last-wins and v2 resolves first-wins—the very asymmetry of Section 9. **Seven slugs disagree, and all seven are slug collisions:**

Table 30: The seven slugs whose framework label differs between the v1 graph JSON's own `framework` field and the v2 re-mine record. All seven appear in the 76-slug colliding population. Source: recount over `corpus/real_world/graphs/` versus `corpus/real_world/metadata_v2.json`.

Slug	v1 graph	v2 record
`Narwhal-Lab__MagicSkills__model`	crewai	autogen
`Saimoguloju__AI-Agents__sample_agent`	autogen	langgraph
`ed-donner__agents__agent`	langgraph	crewai
`ed-donner__agents__main`	langgraph	crewai
`feixiao__ai__chap06`	autogen	crewai
`feixiao__ai__chap07`	autogen	crewai
`martimfasantos__ai-agents-frameworks__00_hello_world`	crewai	langgraph

The net displacement is AutoGen −2, CrewAI +2, which is exactly the spurious 359/113 → 357/115 shift that Section 7.5 retracts. **Blast radius, checked rather than assumed:** the bug touches only this one `Counter`. The fidelity sets take their framework labels from a different map (built from the validation sample's own records), so every number in Sections 3 and 9—the $n = 106$, $n = 115$ and $n = 119$ fidelity tables, the drop-bias table, the re-keying analysis, and the structural-flag counts—is *bit-identical* before and after the fix, which we verified by re-reading the regenerated artifact field-by-field. Five of the seven mislabelled slugs are not in the 119-graph validation sample at all, and the other two are among the 9 colliding graphs already excluded from the $n = 106$ set. The fix is committed and the artifact regenerated.

Remark 6 (Why we report this)**.** *Both defects are instances of one failure: a non-unique key (⟨repo⟩___⟨basename⟩) joined across two datasets that resolve it by opposite tie-break rules. The first cost us 58 comparison pairs; the second cost us a false "correction" in our own audit that we then propagated into a draft of this supplement. Neither was caught by a test, because both produce plausible numbers that sum correctly.*

9.6 Corpus-level drops, for completeness

Independently of collisions, 18 of the 922 v1 graphs have no v2 counterpart: 13 because the repository became unreachable at re-mining time and 5 because re-extraction produced no graph. By framework: AutoGen 14, ADK 2, LangGraph 1, CrewAI 1; 13 of the 18 come from the single repository `Sidreyas/The_Grand_AI_Repo`. This yields the matched corpus of $n = 904$. There are **zero** orphan v2 graphs absent from v1. Source: `matched_fidelity.json` → `drops`.

10 Positioning Against Related Measurement and Enforcement Work

Referenced from `sections/05_related.tex`, which states that the supplement places these efforts along six axes. Table 31 is that placement; it was moved here from the main paper to meet the page limit.

The six axes are **corpus** (what was studied, and how much of it), **frameworks** (breadth of agent frameworks covered), **model** (the formal object, if any, that analysis runs over), **properties** (what is checked or catalogued), **validation** (how the findings were checked), and **guarantees** (what is claimed to follow). The split the table makes visible is that prior measurement studies characterise *which* defects occur, and prior enforcement systems address *how* to constrain agents at runtime, while neither line measures the fidelity of the extracted model on which any static check depends.

Table 31: Positioning against measurement and enforcement work for LLM agents, along the six axes of Section 10. Ours measures the fidelity of the extracted model on which static checks depend, separates distinct audit quantities, and makes soundness conditional on caller-asserted event coverage.

Work	Corpus	Frameworks	Model	Properties	Validation	Guarantees
Ning et al. [6]	dev discussions	agnostic	—	defect taxonomy	manual	none
Xue et al. [10]	∼1,026 bugs	orchestration	—	9 root causes	manual	none
Shen et al. [7]	221 vulns	agent apps	—	14 vuln. types	manual	none
Wang et al. [9]	5,399 progs	cross-framework	dep. graph	security patterns	—	none (unvalidated)
Wang et al. [8]	case studies	runtime	rule DSL	safety rules	case studies	runtime enforcement
Kamath et al. [4]	case studies	generation-time	temporal aut.	temporal constraints	conformance	conformance (runtime)
Ours	922 graphs, 252 repos	4 (cross)	graph + LTL_f	struct. + policy quantities	LLM-recon.; human validation pending	caller-asserted soundness (rel. authored abstraction)

On our own "Guarantees" entry. The wording is deliberately weaker than an unqualified soundness claim, and is the same claim Section 2 proves. What the system delivers is soundness *relative to an authored abstraction* whose event coverage the caller asserts; exact provenance qualifies witnesses but never discharges that assertion (Proposition 1). Reading the cell as "verified soundness" would overstate the result in exactly the way Section 2.5 warns against. Two neighbouring cells are similarly hedged on purpose: our "Validation" entry records that the reference graphs are LLM reconstructions and reports the status of independent human validation, and our "Corpus" entry counts *extracted file-level graphs*, not executable workflow roots or deployed systems—so it is not directly comparable to a bug count or a vulnerability count in the rows above.

What the table does not encode. It is a capability placement, not a benchmark: no row was re-run by us, and the corpus sizes are not commensurable across rows (bug reports, vulnerability records, programs, and extracted graphs are different units). We make no claim of the form "ours is $N\times$ larger".

11 Independent Human Validation

Two-annotator validation has not yet been executed. The locked samples, independent worksheets, annotation guide, and scoring script are included under `corpus/annotations/` and `scripts/human_agreement.py`. Running the latter with `-latex` updates this section and the main paper from the same generated macro file. Until then, LLM-assisted labels are exploratory and this remains the principal internal-validity gap.

12 Known Scope Boundaries and Remaining Gaps

Collected in one place so a reviewer need not reconstruct them.

1. **The effect/capability ontology is not populated by any extractor, and so has no per-framework extraction mapping** (Section 1.3). The fields exist in the model and are populated only when the curated corpus is authored, from declared tool names, by a framework-independent keyword rule. `state_update` is never assigned by any code path. `back_edge` is set only by `build_corpus.py`. *The main paper's Section 2 now states this explicitly, and the two documents agree*: what Table 3 documents is the per-framework mapping into the exclusive `NodeKind`/`EdgeKind` view, which is the only one that runs during extraction.

2. **Event-trace conservatism (H1) is assumed, not verified** (Section 2.5). Theorem 1 is conditional; provenance metadata qualifies witnesses and only the caller's assertion discharges the hypothesis. A body-level effect analysis is the missing piece.

3. **Per-framework extractor accuracy "on stubs" is not available** (Section 6.2). `corpus/ground_truth/` does not exist and the available substitute is circular. Real-code accuracy is reported instead.

4. **No quantitative Promela/SMV modeling-effort measurement exists** (Section 6). One hand-written Promela model is committed; no timing, no lines-of-model count, no head-to-head run.

5. **Confidence-output consistency has been repaired.** The submitted `confidence_intervals.json` is regenerated on the matched, collision-free $n = 106$ fidelity set with repository-clustered bootstrap intervals and the self-repository-excluded 186-flag triage set. It also records HGP-1 as 12/119. The end-to-end consistency check rejects the superseded $n = 120/187$ output.

6. **HGP-1 supersedes the earlier human-gate estimand** (Section 8). The primary figure is 12/119; the earlier 3/119 is withdrawn, not averaged in. *Residual gap:* HGP-1 is single-rater, 11 cases remain `arguable` for want of a repository-level corpus. The 12/119 count is conditional on the upstream screen: the 87 workflows with no source-level side-effecting tool were not re-read *under HGP-1*—they were excluded by the same classification pass that produced the 32, and an independent effect-signature sweep of all 87 flagged 10 for manual re-reading, of which 9 are correctly screened and 1 would be `arguable`. Screening error and label error can move the observed count in either direction.

7. **Absolute benchmark timings are not comparable across drafts** (Section 6). Three recorded figures at 5,000 nodes span a factor of ≈33 across machines. Only the asymptotic shape is a property of the system.

8. **Independent human-validation status: human validation pending** (Section 11). Reference graphs and triage labels originate in LLM passes sharing a model family. The survival test of Section 5 weakens, but does not by itself remove, this threat: it reproduces the headline decomposition from graphs alone without consulting the labels (93.0% agreement), which is evidence against label *dependence*, not evidence of label *correctness*.

9. **The reference graphs are not a semantic oracle, and we now measure by how much** (Sections 8, 5). Node kinds are assigned from naming and framework type, so a `human` node need not be able to block; 5/186 flags trip the validity guard for this reason, and 9 of the 12 source-audited violations are invisible even to the reconstruction because the effect lives in a node body. Every human-gate estimand computed over reference graphs is therefore biased downward by an amount we bound but cannot correct.

References

[1] Alessandro Cimatti, Edmund Clarke, Enrico Giunchiglia, et al. NuSMV 2: An opensource tool for symbolic model checking. In *International Conference on Computer Aided Verification (CAV)*. Springer, 2002.

[2] Giuseppe De Giacomo and Moshe Y. Vardi. Linear temporal logic and linear dynamic logic on finite traces. In *International Joint Conference on Artificial Intelligence (IJCAI)*, 2013.

[3] Gerard J. Holzmann. The model checker SPIN. *IEEE Transactions on Software Engineering*, 23(5):279–295, 1997.

[4] Adharsh Kamath, Sishen Zhang, Calvin Xu, Shubham Ugare, Gagandeep Singh, and Sasa Misailovic. Enforcing temporal constraints for LLM agents. Preprint, 2026. `https://github.com/structuredllm/agent-c`.

[5] Daniel Kroening and Michael Tautschnig. CBMC – C bounded model checker. In *International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*. Springer, 2014.

[6] Kaiwen Ning, Jiachi Chen, Jingwen Zhang, Wei Li, Zexu Wang, Yuming Feng, Weizhe Zhang, and Zibin Zheng. Defining and detecting the defects of the large language model-based autonomous agents. *arXiv preprint arXiv:2412.18371*, 2024.

[7] Zhuoxiang Shen, Jiarun Dai, Yuan Zhang, and Min Yang. Security debt in LLM agent applications: A measurement study of vulnerabilities and mitigation trade-offs. In *IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2025.

[8] Haoyu Wang, Christopher M. Poskitt, and Jun Sun. AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents. *arXiv preprint arXiv:2503.18666*, 2025.

[9] Shenao Wang, Xinyi Hou, Yanjie Zhao, Xiao Cheng, and Haoyu Wang. AgentFlow: Building agent dependency graphs for static analysis of agent programs. arXiv:2607.01640, 2026.

[10] Ziluo Xue, Yanjie Zhao, Shenao Wang, Kai Chen, and Haoyu Wang. A characterization study of bugs in LLM agent workflow orchestration frameworks. In *IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2025.